Chiron

Research & Decision Narrative (Ideation Stage)

Progress Report 1 • 29 Sep 2025 — 12 Oct 2025

Master’s Research Progress Update

Fahad Alothman

Progress Report 1

Title: Research & Decision Narrative (Ideation Stage)
Reporting Window: 29 Sep 2025 — 12 Oct 2025
Submission Context: Master’s Research Progress Update

Abstract

This first progress report documents the research and ideation stage of the project, with emphasis on how the initial problem framing evolved under evidence. The report does not present a mature product architecture; instead, it captures the early decision process that shaped the project’s direction. The core issue identified in this period was that common agentic coding workflows were heavily chat-centric and required developers to carry a large implicit mental model of system decisions, constraints, and unfinished assumptions. This made long-horizon software delivery difficult to govern and hard to reproduce.

The report describes an early landscape analysis of agentic development approaches, including chat-first tooling, emerging spec-driven workflows, and framework-based orchestration methods. It also documents why BMAD was selected as an initial baseline for structured method execution, and why the project narrowed toward software development lifecycle (SDLC) guidance rather than immediately pursuing full computer-use/sandboxing infrastructure as the primary implementation track.

This report therefore establishes the decision foundation for later reports: first understand and structure guidance for decision quality in software projects, then progressively evaluate how runtime orchestration and optimization techniques can be integrated in later phases.

---

1. Problem Framing at the Start of the Project

At the start of this reporting period, the project question was not “how to generate more code with AI.” The guiding question was:

> How can developers make better, more reliable implementation decisions when using agentic systems over multi-step software lifecycles?

The practical concern behind this question was cognitive overhead. In many chat-centric workflows, a developer must remember:

- previous architectural decisions,
- pending constraints,
- tradeoffs already accepted,
- artifacts that are still drafts,
- and which assumptions are validated versus speculative.

In short sessions, this is manageable. Across longer implementation arcs, it becomes brittle. When context is fragmented across chat turns, quality depends on individual memory rather than explicit system structure.

This early observation shaped the first major thesis direction:

1. **The issue is not only generation quality.**
2. **The issue is lifecycle guidance quality.**
3. **Decision traceability must be treated as a first-class requirement.**

---

2. Research Scan: Agentic Development Evolution

During this period, the project examined the broader evolution of agentic development practices and tools.

2.1 Dominant Early Pattern: Atomic Chat Sessions

The baseline mode in many AI coding experiences was still atomic or short-horizon chat loops: prompt, response, patch, repeat. This pattern has high speed and low setup cost, but weak continuity for long-running software initiatives.

Key pitfall observed:

- the development process can become “chat-log deep” but “system-model shallow.”

2.2 Next Step in the Ecosystem: Spec-Driven Approaches

Spec-driven approaches emerged to externalize intent and reduce ambiguity before code generation. This was important because it moved from implicit instructions toward explicit requirements, acceptance conditions, and task decomposition. During this period, the relevant signal was less a single dominant product and more a broader movement toward making intent explicit before implementation [4], [5].

2.3 Workflow + Spec Hybridization

A second evolution combined specs with execution workflows. Instead of treating specification as static documentation, these approaches used spec artifacts to drive ordered implementation actions.

This hybrid pattern was particularly relevant to this project because it aligned with the central problem: maintaining a coherent decision model over time.

2.4 Structured Method Baseline (BMAD)

BMAD was reviewed as an AI-driven development framework that organizes work from ideation through implementation using specialized agents and guided workflows [1]–[3]. This provided a useful baseline for early experimentation because it reduced ad hoc prompting behavior and introduced process structure.

---

3. Why BMAD Was Chosen as the Initial Baseline

BMAD was chosen in the ideation stage for methodological reasons, not as an end-state product choice [1]–[3].

3.1 What BMAD Provided Early

- phased workflow structure,
- role-specialized agent responsibilities,
- explicit decomposition from idea to stories/tasks,
- repeatable collaboration pattern between human decisions and agent execution.

3.2 What BMAD Is (and Is Not) in This Report

For this report, BMAD is treated as:

- **a development framework baseline** for structured AI-assisted delivery,
- **not** a replacement for programming tools,
- **not** a complete runtime governance system by itself,
- **not** the final architectural endpoint of this project.

This distinction is important: the project used BMAD as a disciplined starting methodology to expose where additional guidance layers would eventually be required.

---

4. Parallel Paradigms Considered in Research Stage

The early research did not assume a single paradigm. Two major paradigms were considered in parallel:

1. **SDLC Guidance Paradigm**
   Focus: better planning/execution decisions, clearer lifecycle progression, explicit rationale and traceability.

2. **Computer-Use / Sandboxing Paradigm**
   Focus: safe autonomy, permission boundaries, execution isolation, and managed operational risk.

4.1 Why This Distinction Matters

Both paradigms are important, but they optimize different failure modes:

- SDLC guidance reduces decision ambiguity and process drift.
- sandboxing/computer-use reduces unsafe autonomy and execution risk.

During this first reporting period, the project deliberately prioritized the SDLC guidance direction as the primary thesis lane, while retaining computer-use/sandboxing as an adjacent strategic concern for later reports.

This was a scope discipline decision: tackling both deeply at once would reduce clarity and execution quality in the early stage.

---

5. Assumptions, Validation Attempts, and Corrections

This section records the key assumptions in the ideation stage and how they were tested.

Assumption A

Initial assumption: Better coding models/tools alone will materially solve long-horizon delivery quality.
What challenged it: Repeated evidence that context continuity and rationale traceability remain fragile in chat-centric workflows.
Correction: Treat guidance structure as a first-class system problem, not a side effect of stronger models.

Assumption B

Initial assumption: Spec artifacts are enough if written clearly.
What challenged it: Specs without execution discipline can still drift in practice.

Correction: Couple specification with workflow discipline and explicit decision checkpoints.

Assumption C

Initial assumption: One paradigm can be optimized first and reused unchanged for all concerns.
What challenged it: Safety/governance concerns in autonomous execution are distinct from SDLC guidance concerns.
Correction: Keep paradigms explicit: prioritize SDLC guidance first, track sandboxing paradigm evolution in parallel.

Assumption D

Initial assumption: A framework baseline is equivalent to a final architecture.
What challenged it: Frameworks provide method scaffolding but do not automatically materialize project-specific guidance semantics.
Correction: Use framework baseline to accelerate learning; progressively introduce project-specific constructs in later stages.

---

6. Timeline (Progress Report 1 Scope)

Week 1 (Research Framing and Landscape Mapping)

- Defined the project's research focus as decision quality in AI-assisted software delivery.
- Mapped dominant workflow patterns from chat-centric use toward spec/workflow structured methods.
- Identified key candidate tool/method references for comparative analysis.

Week 2 (Baseline Selection and Scope Narrowing)

- Adopted BMAD as the initial methodological baseline for structured ideation/planning.
- Distinguished SDLC guidance work from computer-use/sandboxing work.
- Established progressive disclosure plan for reports:
- early reports: research + ideation findings,
- later reports: deeper execution/orchestration decisions as they actually occurred.

---

7. Tool and Method Context (Early-Stage Positioning)

The early tool and method context reviewed in this period included:

- chat-centric coding agents,
- spec-driven development approaches,
- early parallel coding-agent orchestration products such as Conductor.build [6], [7],
- workflow orchestration patterns,
- structured methodology frameworks (including BMAD),
- and rapidly evolving ecosystem discussions around safety and sandboxed autonomy.

This context helped establish an important narrative rule for future reports:

> Do not overstate maturity too early. Report findings progressively as they were discovered and validated.

That rule is intentionally applied here: this report remains within research/ideation scope and does not attribute later-stage architecture realizations to this first period.

Conductor is especially useful to place correctly at this stage. It represented a concrete product expression of the parallel-agent, isolated-workspace, git-worktree-oriented paradigm that Chiron was still seriously evaluating [6], [7]. However, this report should also make clear that the master's-project thesis lane was not simply to build another conductor for coding agents. The primary lane remained SDLC guidance and reducing the cognitive burden of carrying an implicit project model, even while orchestration and sandboxing stayed active adjacent concerns.

---

8. Decisions Produced by Progress Report 1

By the end of this reporting window, the following decisions were established:

1. The project will be framed around **guidance for SDLC decision quality** as the primary research objective.
2. BMAD will be used as a **structured baseline method** in early stages.
3. The computer-use/sandboxing paradigm will be monitored and referenced, but not made the primary implementation target at this stage.
4. Future reports will use **progressive disclosure**: each report should represent what was actually known and implemented in that period, not retroactive coherence from later insights.

---

9. Risks and Open Questions Entering Progress Report 2

Risks

- Overfitting the process to documentation rather than execution reality.

- Conflating framework guidance with project-specific guidance requirements.
- Attempting to solve too many paradigms simultaneously in early-stage development.

Open Questions

- How should guidance quality be measured in practice (consistency, traceability, recoverability)?
- Which aspects of spec-driven process must become executable versus remain advisory?
- What is the minimum structure needed to reduce cognitive load without introducing process overhead?

These open questions define the handoff into Progress Report 2.

---

10. Conclusion

Progress Report 1 documents a research-stage outcome: the project moved from broad exploration to a focused thesis direction. The key result is not a finalized architecture, but a clarified problem model and a justified scope strategy.

The period established that agentic software development quality depends on more than code generation speed. It depends on how well a system preserves intent, constraints, rationale, and progression across time. This insight motivated selection of a structured baseline method and a deliberate focus on SDLC guidance while acknowledging, but temporarily de-prioritizing, deeper sandboxing/computer-use implementation concerns.

This creates a rigorous foundation for subsequent progress reports, where assumptions and implementation choices can be introduced in the order they were actually discovered.

References

[1] bmad-code-org, “BMAD-METHOD,” GitHub repository, 2025. [Online]. Available: https://github.com/bmad-code-org/BMAD-METHOD
[2] bmad-code-org, “BMAD Method v4.44.1,” GitHub release, Sep. 29, 2025. [Online]. Available: https://github.com/bmad-code-org/BMAD-METHOD/releases/tag/v4.44.1
[3] V. Mysore, “What is BMAD-METHOD™? A Simple Guide to the Future of AI-Driven Development,” Medium, Sep. 8, 2025. [Online]. Available: https://medium.com/@visrow/what-is-bmad-method-a-simple-guide-to-the-future-of-ai-driven-development-412274f91419
[4] R. Doradla, “Beyond Vibe Coding: Amazon Introduces Kiro, the Spec-Driven Agentic AI IDE,”

InfoQ, Aug. 18, 2025. [Online]. Available: https://www.infoq.com/news/2025/08/aws-kiro-spec-driven-agent/
[5] Kiro, “From chat to specs: a deep dive into AI-assisted development with Kiro,” Jul. 15, 2025. [Online]. Available: https://kiro.dev/blog/from-chat-to-specs-deep-dive
[6] Conductor, “Run a team of coding agents on your Mac,” 2025. [Online]. Available: https://conductor.build/
[7] Conductor, “Run a team of coding agents,” Documentation, 2025. [Online]. Available: https://docs.conductor.build/